

Lecture 1

Number Systems

Binary, octal and hexadecimal · base conversion · complements
· signed numbers and overflow

Reading: Mano & Ciletti, *Digital Design with an Introduction to the Verilog HDL*, 6th ed. — Chapter 1, §1.1–1.6

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
1	0	1	1	0	1	0	1

$$= 128 + 32 + 16 + 4 + 1 = 181$$

$$(181)_{10} = (B5)_{16} = (265)_8$$

Today's roadmap

1

§1.2

Number bases

Any base, one formula

2

§1.3

Base conversion

Divide, multiply, regroup

3

§1.5

Complements

Subtraction by addition

4

§1.6

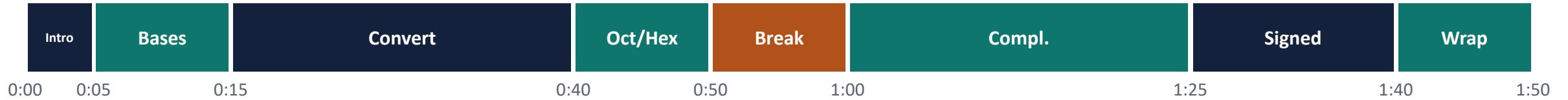
Signed numbers

2's complement and overflow

By the end of class you can...

- Convert between decimal, binary, octal and hexadecimal
- Convert integers and fractions separately
- Form r 's and $(r-1)$'s complements
- Subtract using complements
- Represent signed numbers and detect overflow

Agenda for today's session



Elapsed time	Topic	In-class work
0:00 – 0:05	Welcome, roadmap, digital systems	—
0:05 – 0:15	Positional notation in any base	Example 1
0:15 – 0:40	Decimal to any base: integers and fractions	Examples 2, 3 · Your turn 1
0:40 – 0:50	Octal and hexadecimal by grouping	Example 4
0:50 – 1:00	Break	
1:00 – 1:25	Complements and subtraction	Examples 5, 6
1:25 – 1:40	Signed numbers and overflow	Example 7 · Your turn 2
1:40 – 1:50	Concept check and wrap-up	Homework preview

Digital systems count in binary

§1.1

Why two values?

Electronic circuits are most reliable when a signal only has to be HIGH or LOW. Each two-valued digit is a **bit**.

Positional notation, base r

$$(a_n \dots a_1 a_0 . a_{-1} \dots a_{-m})_r = \sum a_i r^i$$

Each digit a_i lies in $0 \dots r - 1$ and is weighted by a power of the base.

Decimal	Binary	Octal	Hex
0	0000	0	0
1	0001	1	1
2	0010	2	2
3	0011	3	3
4	0100	4	4
5	0101	5	5
6	0110	6	6
7	0111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

Example 1 — Any base to decimal

§1.2

EXAMPLE

Convert $(11010.11)_2$ and $(4021.2)_5$ to decimal.

2^4	2^3	2^2	2^1	2^0		2^{-1}	2^{-2}
1	1	0	1	0	.	1	1
16	8	4	2	1		$\frac{1}{2}$	$\frac{1}{4}$

Weight each digit by its place value, then add:

$$16 + 8 + 0 + 2 + 0 + \frac{1}{2} + \frac{1}{4} = 26.75$$

The point separates non-negative from negative powers.

- 1 Write the weight of each place: 16,8,4,2,1, $\frac{1}{2}$, $\frac{1}{4}$
- 2 Keep the weights under the 1s: 16,8,2, $\frac{1}{2}$, $\frac{1}{4}$
- 3 Add them: $(11010.11)_2 = \mathbf{26.75}$
- 4 Base 5 weights: 125,25,5,1,0.2
- 5 Multiply: $4 \cdot 125 = 500$, $0 \cdot 25 = 0$, $2 \cdot 5 = 10$
- 6 and $1 \cdot 1 = 1$, $2 \cdot 0.2 = 0.4$
- 7 Add them: $(4021.2)_5 = \mathbf{511.4}$

Decimal to any base: integers

§1.3

EXAMPLE

Example 2 — Convert $(41)_{10}$ to binary and $(153)_{10}$ to octal.

Repeated division by 2

Divide	Quot.	Rem.
$41 / 2$	20	1 (LSB)
$20 / 2$	10	0
$10 / 2$	5	0
$5 / 2$	2	1
$2 / 2$	1	0
$1 / 2$	0	1 (MSB)

Read the remainders from the last one up:
 $(101001)_2$

Repeated division by 8

Divide	Quot.	Rem.
$153 / 8$	19	1
$19 / 8$	2	3
$2 / 8$	0	2

Read upward: $(231)_8$

Why it works

Dividing by r shifts the number one place right; the remainder is the digit that fell off.

So the FIRST remainder is the least significant digit.

Check: $2 \cdot 64 + 3 \cdot 8 + 1 = 153 \checkmark$

Decimal to any base: fractions

§1.3

EXAMPLE

Example 3 — Convert $(0.6875)_{10}$ to binary.

Multiply by 2	Result	Digit
0.6875×2	1.375	1 (first)
0.375×2	0.75	0
0.75×2	1.5	1
0.5×2	1.0	1 (last)

Read the integer parts from the top down:

$$(0.6875)_{10} = (0.1011)_2$$

- 1 Multiplying by r shifts one place left: the integer part is the next digit.
- 2 Keep the fraction part and repeat.
- 3 Stop at 0 — or at the precision you need.
- 4 Check: $\frac{1}{2} + \frac{1}{8} + \frac{1}{16} = 0.6875 \checkmark$
- 5 A number with both parts: convert the integer and the fraction separately, then join them.

Your turn 1 — Conversions

§1.3

YOUR TURN

≈ 10 min

- (a) Convert $(247.375)_{10}$ to binary.
- (b) Use your answer to write the number in octal and in hexadecimal.
- (c) Convert $(4F.A)_{16}$ to decimal.

Check each answer by converting back.

Your turn 1 — Solution (a)

§1.3

SOLUTION

Integer part: divide by 2

Divide	Quot.	Rem.
247 / 2	123	1 (LSB)
123 / 2	61	1
61 / 2	30	1
30 / 2	15	0
15 / 2	7	1
7 / 2	3	1
3 / 2	1	1
1 / 2	0	1 (MSB)

Fraction: multiply by 2

Multiply	Result	Digit
0.375×2	0.75	0
0.75×2	1.5	1
0.5×2	1.0	1

- 1 Divide 247 by 2 until the quotient is 0.
- 2 Read the remainders from the bottom up: 11110111
- 3 Multiply 0.375 by 2; keep the fraction part each time.
- 4 Stop at 0. Read the digits from the top down: .011
- 5 Join the parts: $(11110111.011)_2$

Your turn 1 — Solution (b), (c)

§1.3

SOLUTION

(b) Group from the binary point

0	1	1	1	1	0	1	1	1	.	0	1	1
3			6			7					3	

1	1	1	1	0	1	1	1	.	0	1	1	0
F				7					6			

Octal groups of 3 (top), hex groups of 4 (bottom); pad with 0s at the outer ends.

- 1 (b) Octal: groups of 3 $\rightarrow$ 011 110 111 . 011
- 2 Each group is one digit: $(367.3)_8$
- 3 Hex: groups of 4 $\rightarrow$ 1111 0111 . 0110
- 4 Each group is one digit: $(F7.6)_{16}$
- 5 (c) Weights of $4F.A$: $16, 1, 1/16$
- 6 $4 \cdot 16 = 64$; $F = 15$, so $15 \cdot 1 = 15$
- 7 $A = 10$, so $10/16 = 0.625$
- 8 Add: $64 + 15 + 0.625 = \mathbf{79.625}$

Octal and hexadecimal by grouping

§1.4

EXAMPLE

Example 4 — Write $(10110001101011.111100000110)_2$ in octal and in hexadecimal.

Groups of three

$= (26153.7406)_8$

010	110	001	101	011	.	111	100	000	110
2	6	1	5	3	.	7	4	0	6

Group outward from the binary point; pad the outer ends with 0s.

Groups of four

$= (2C6B.F06)_{16}$

0010	1100	0110	1011	.	1111	0000	0110
2	C	6	B	.	F	0	6

Break

10 minutes — we resume at the 1:00 mark

Next: complements and subtraction

Complements

§1.5

Two complements for every base

Diminished radix, $(r - 1)$'s complement of an n -digit N :

$$(r^n - 1) - N$$

Radix, r 's complement:

$$r^n - N \text{ (and 0 for } N = 0\text{)}$$

So the r 's complement is the $(r - 1)$'s complement plus 1.

Number	9's / 1's	10's / 2's
546700	453299	453300
012398	987601	987602
1101100	0010011	0010100
0110111	1001000	1001001

Shortcuts in binary

1's complement: invert every bit.

2's complement: invert every bit and add 1 — or copy up to and including the first 1 from the right, then invert the rest.

Example 5 — Subtraction by complement

§1.5

EXAMPLE

Compute $X - Y$ and $Y - X$ for $X = 1010100$ and $Y = 1000011$ using 2's complements.

$X - Y$: add the 2's complement of Y

X	1	0	1	0	1	0	0
$+2's \text{ of } Y$	0	1	1	1	1	0	1
sum	1	0	0	1	0	0	1

The rule

End carry: discard it; the result is positive.

No end carry: the result is negative. Take the 2's complement of the sum and put a minus sign in front.

- 1 2's comp. of Y : invert $\rightarrow 0111100$; add 1 $\rightarrow 0111101$
- 2 Add: $1010100 + 0111101 = 1\ 0010001$
- 3 End carry is 1: discard it. $X - Y = 0010001$
- 4 2's comp. of X : invert $\rightarrow 0101011$; add 1 $\rightarrow 0101100$
- 5 Add: $1000011 + 0101100 = 1101111$
- 6 No end carry, so $Y - X$ is negative.
- 7 2's comp. of 1101111 : $0010000 + 1 = 0010001$
- 8 $Y - X = -0010001$
- 9 Check: $84 - 67 = 17 = (10001)_2 \checkmark$

Example 6 — With the 1's complement

§1.5

EXAMPLE

Repeat $X - Y$ ($X = 1010100$, $Y = 1000011$) with the 1's complement.

X	1	0	1	0	1	0	0
$+1's \text{ of } Y$	0	1	1	1	1	0	0
sum	1	0	0	1	0	0	0
$+ \text{ carry}$	0	0	1	0	0	0	1

The end carry is added back in (end-around carry): the answer is the same, 0010001.

- 1's complement of Y : invert every bit $\rightarrow 0111100$
- Add: $1010100 + 0111100 = 1\ 0010000$
- There is an end carry of 1. Remove it: 0010000
- Add it at the LSB: $0010000 + 1 = 0010001$
- Same answer as Example 5 ✓
- Cost: the extra addition. That is why hardware uses the 2's complement.

Signed binary numbers

§1.6

Decimal	Signed 2's	Signed 1's	Signed magnitude
+3	0011	0011	0011
+2	0010	0010	0010
+1	0001	0001	0001
0	0000	0000	0000
-0	—	1111	1000
-1	1111	1110	1001
-2	1110	1101	1010
-3	1101	1100	1011
-4	1100	—	—

Signed 2's complement

The leftmost bit is the sign (0 +, 1 -).

n bits hold -2^{n-1} to $2^{n-1} - 1$

One zero, and addition works with no special cases.

Reading a negative number

Take the 2's complement to find the magnitude:
 $1101 \rightarrow 0011$, so $1101 = -3$.

Example 7 — Signed sums and overflow

§1.6

EXAMPLE

Add $(+100) + (+50)$ and $(+83) + (-45)$ in 8-bit signed 2's complement.

+100	0	1	1	0	0	1	0	0
+50	0	0	1	1	0	0	1	0
sum	1	0	0	1	0	1	1	0

Sign bit is 1, so the sum reads as -106 , not $+150$.

Carry INTO the sign bit = 1, carry OUT = 0:

$$V = C_{in} \oplus C_{out} = 1 \rightarrow \text{overflow}$$

- 1 $+100 = 01100100$ and $+50 = 00110010$
- 2 Add column by column: 10010110
- 3 Sign bit is 1: a negative result from two positives.
- 4 $+150$ is above $+127$, the 8-bit limit: **overflow**
- 5 $+83 = 01010011$
- 6 $45 = 00101101 \rightarrow$ invert $11010010 \rightarrow$ add 1: $-45 = 11010011$
- 7 Add: $01010011 + 11010011 = 1\ 00100110$
- 8 Discard the end carry: $00100110 = +38$ ✓
- 9 Opposite signs can never overflow.

Your turn 2 — Signed arithmetic

§1.6

YOUR TURN

≈ 10 min

- (a) Write -45 and -98 in 8-bit signed 2's complement.
- (b) Add them. What 8-bit result does the hardware produce?
- (c) Does overflow occur? Justify it two ways.
- (d) What is the range of a 12-bit signed 2's-complement number?

Your turn 2 — Solution

§1.6

SOLUTION

-45	1	1	0	1	0	0	1	1
-98	1	0	0	1	1	1	1	0
sum	1	0	1	1	1	0	0	1

Discard the end carry: the result is $01110001 = +113$.

Carry into the sign bit = 0, out of it = 1.

Check: $-143 + 256 = 113$

- 1 (a) $45 = 32 + 8 + 4 + 1 = 00101101$
- 2 Invert 11010010 , add 1: $-45 = \mathbf{11010011}$
- 3 $98 = 64 + 32 + 2 = 01100010$
- 4 Invert 10011101 , add 1: $-98 = \mathbf{10011110}$
- 5 (b) Add: $11010011 + 10011110 = 1\ 01110001$
- 6 Keep 8 bits: $\mathbf{01110001} = +113$
- 7 (c) Two negatives gave a positive: overflow.
- 8 Carry in to sign bit 0, out 1: $V = 0 \oplus 1 = 1 \checkmark$
- 9 (d) $n = 12$: -2^{11} to $2^{11} - 1 = \mathbf{-2048\ to\ +2047}$

Concept check

1

Why does the first remainder in repeated division give the LSB?

2

Why can octal and hex be converted to binary with no arithmetic?

3

What is the 2's complement of 00000000 in 8 bits, and why is that good?

4

Can adding a positive and a negative number overflow?

Key takeaways

Positional value

$$\sum a_i r^i$$

Integer $\rightarrow$ base r

divide, read up

Fraction $\rightarrow$ base r

multiply, read down

Octal / hex

3 / 4 bits per digit

r 's complement

$$r^n - N$$

Signed overflow

$$V = C_{in} \oplus C_{out}$$

Every number in a digital system is one of these representations.

Homework #1 due at the start of Lecture 3

Next: binary codes, parity and binary logic